OpenZeppelin | security

Rate Limiter Audit

June 11, 2026

Table of Contents

Summary

Type	Library
Timeline	From 2026-05-28 To 2026-05-29
Languages	Move

Total Issues	6 (6 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	0 (0 resolved)
Notes & Additional Information	6 (6 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the OpenZeppelin/contracts-sui repository at commit 9cd0672.

In scope were the following files:

```
contracts
└── utils
    └── sources
        └── rate_limiter.move
```

***Update**: The fixes for the findings highlighted in this report have all been merged at commit 36f2787.*

System Overview

The `rate_limiter` module is part of `openzeppelin_utils`, a collection of embeddable primitives for Sui Move development. It provides a single rate-limiting primitive, `RateLimiter`, that integrators embed as a field inside their own objects to throttle the rate at which a sensitive operation can be invoked. There is no registry, policy object, or separate identifier to track and assert against: the limiter's scope is whatever object holds it, and its authority extends no further than the call sites that mutate it.

`RateLimiter` is a `store + drop` enum exposing three interchangeable strategies behind one API:

- `Bucket`: a continuously refilling token bucket that credits `refill_amount` units every `refill_interval_ms`, capped at `capacity`. It permits sustained throughput with short bursts up to the cap.
- `FixedWindow`: allows up to `capacity` units per fixed-length window anchored at a chosen start time, resetting to full at each boundary. Internally it is handled as a bucket whose single-window refill equals its full capacity.
- `Cooldown`: allows up to `capacity` units to be drawn before gating for `cooldown_ms`, after which the full `capacity` becomes available again.

The processing flow is uniform across variants:

- Integrators construct a limiter with one of the `new_bucket`, `new_fixed_window`, or `new_cooldown` constructors and store it on their object.
- On hot paths they call `consume_or_abort` or `try_consume`, both of which project time-based state forward (bucket accrual, window rollover, or cooldown release) before deciding whether the request fits. The projection is all-or-nothing: it is committed to stored state only when the consume succeeds, and a refused `try_consume` leaves the limiter untouched.
- Read paths call `available`, which projects pending transitions on read without mutating state.

Time is sourced exclusively from the Sui `Clock`, whose monotonicity the module relies on to keep its anchor projections from underflowing.

The module deliberately omits in-place reconfiguration. To change a limiter's configuration or runtime state, the integrator reads the current values through the getters, constructs a fresh `RateLimiter`, and overwrites the field. Constructors validate structural invariants (positive configuration values, an initial balance not exceeding `capacity`, anchors not set in the future, and the mutually exclusive cooldown gate-and-balance pairing), while the choice of reconfiguration semantics is left entirely to caller code.

Security Model and Trust Assumptions

The module defines no privileged on-chain roles of its own. It is an unopinionated primitive that enforces structural validity and the arithmetic of each strategy, and delegates all authorization and policy decisions to the integrating code. The security-relevant actors are therefore the integrator who embeds and operates the limiter, and the operator who selects its configuration.

- **Integrator**: embeds `RateLimiter` in their own object and decides which call sites may mutate it. Any function taking a mutable reference to the limiter advances live state, so the integrator is responsible for gating the entry functions that expose `consume_or_abort`, `try_consume`, and reconstruction behind an authorization model appropriate to the call site (capability, access control, governance, multisig, or similar). The module performs no such checks itself. Because reconfiguration is carried out by reading current state and overwriting the field with a freshly constructed limiter, the integrator also chooses the reconfiguration policy; the library validates only that the new value is structurally well-formed, not that the transition preserves any intended throttling behavior.
- **Operator**: chooses the strategy and the configuration values passed to the constructors, covering the per-strategy parameters (`capacity`, `refill_amount`, `refill_interval_ms`, `window_ms`, `window_start_ms`, `cooldown_ms`) and the initial runtime state (`initial_available` together with the relevant time anchor). Reconfiguration is performed by reading current state, constructing a fresh `RateLimiter`, and overwriting the field, so the operator also owns the policy choice between preserving anchors, full resets, and other transition strategies.

Notes & Additional Information

N-01 `try_consume` Aborts on a Zero Amount, Breaking the `try_*` Non-Aborting Convention

Across the Sui standard library and the wider Move ecosystem, the `try_` prefix means a function returns a value instead of aborting: `vec_map::try_get`, `string::try_utf8`, and the `u64::try_as_u8` family all return `none` rather than reverting. Callers select a `try_*` function precisely for that fail-safe surface.

However, the `try_consume` function breaks this convention. Its first statement asserts `amount > 0` and aborts with `EInvalidAmount` otherwise, so `try_consume(0, ...)` reverts instead of returning `false`. This is especially error-prone when a caller reads `available` and passes the result straight into `try_consume`, since `available` returns zero whenever the limiter is drained, exhausted, or gated.

Consider having `try_consume(0, ...)` return `false` and moving the zero-amount assertion into `consume_or_abort`, where an abort is consistent with that function's contract. This restores the non-aborting guarantee implied by the `try_` prefix while preserving the strict behavior for callers that want it.

Update: *Resolved in pull request #342.*

N-02 Reconfiguring a `Bucket` to a Faster Rate While Preserving the Anchor Mints Retroactive Credit

The module's reconfiguration guidance instructs integrators to rebuild a fresh limiter from the getters, and `new_bucket` documents that an earlier `last_refill_ms` anchor may be passed to "preserve the refill phase when reconstructing under a new configuration."

However, the accrual logic applies the current rate over the entire span since the anchor. When a reconfiguration both changes the rate and preserves the old anchor, time elapsed under the

previous rate is re-priced at the new rate, minting tokens instantly. For example, a bucket of capacity 100 refilling 1 token per 100 milliseconds reports 10 available and a projected anchor of 1000 at timestamp 1050. Carrying over those values while switching to 1 token per 10 milliseconds re-prices the 50-millisecond remainder as 5 tokens, so the bucket reports 15 available at the same timestamp rather than 10.

Consider documenting that the anchor may be preserved only when the rate is unchanged, and that any change to `refill_amount` or `refill_interval_ms` must re-anchor to the current timestamp. The test suite already applies this safe pattern, re-anchoring with the comment "no retroactive new-rate credit," so the constraint is understood internally but absent from the public documentation.

Update: *Resolved in pull request #350.*

N-03 Inconsistent Field Ordering Across Constructors and `Cooldown` Destructure in `rate_limiter`

The `rate_limiter` module exposes three variants (`Bucket`, `FixedWindow`, and `Cooldown`) built through dedicated `new_*` constructors and dispatched on in `try_consume`. Each variant carries a capacity, strategy-specific configuration, an `initial_available` token balance, and a time anchor, so a stable field ordering across constructor signatures and match arms lets a reader compare variants at a glance.

Two related inconsistencies break that convention. The constructors `new_bucket` and `new_cooldown` place `initial_available` ahead of the time anchor, while `new_fixed_window` places `window_start_ms` first. Both slots are `u64`, so the compiler cannot detect a swap; a caller transposing the two trips `EInitialAboveCapacity` rather than `EWindowAnchorInFuture`, since a wall-clock millisecond easily exceeds plausible capacities. The abort fires, but names the wrong invariant and can send an integrator down an incorrect debug path. Additionally, the same convention slips internally: the `Cooldown` arm of `try_consume` destructures as `{ cooldown_ms, capacity, available, cooldown_end_ms }`, while the variant declaration orders the fields `{ capacity, cooldown_ms, cooldown_end_ms, available }`. The `Bucket` and `FixedWindow` arms mirror their declarations; only `Cooldown` diverges.

Consider aligning `new_fixed_window` with the other constructors by placing `initial_available` ahead of `window_start_ms`, and reordering the `Cooldown` destructure in `try_consume` to match its declared field order.

Update: *Resolved in pull request #343.*

N-04 Documentation Improvements

Throughout the codebase, multiple opportunities to improve the documentation were identified:

- The `EWrongVariant` error comment reads "Reconfigure target does not match the limiter's current variant." However, in-place reconfiguration was removed, and this error is now raised only by the variant-typed getters. The comment therefore describes an API that no longer exists and should be updated to reflect the getter-mismatch context.
- The summary line of the `available` function describes the result as "available capacity." The returned value is the available units, or headroom, not the capacity. Furthermore, the summary mentions only accrual and window reset, omitting cooldown release, which appears only in the per-variant breakdown that follows. Consider correcting the wording and listing all three projection types in the summary.
- The `new_fixed_window` constructor does not warn that a backdated anchor silently discards the seeded `initial_available`. When the anchor lies a full window or more in the past, the first read crosses a window boundary and resets the balance to capacity. For example, `new_fixed_window(5, 100, 0, 2)` evaluated at timestamp 100 reports 5, not the seeded 2, because one full window has elapsed since the anchor at 0. Anchoring at 50, still inside the window, preserves the seeded 2. This follows directly from the documented rollover rule, but is not flagged as a seeding hazard. Consider noting that `initial_available` is meaningful only within the window containing the anchor.

Consider applying the changes above to improve the readability and correctness of the codebase documentation.

Update: *Resolved in pull request #347.*

N-05 Absence of Event Emission Is Undocumented in `rate_limiter`

The `rate_limiter` module is an embeddable primitive that shares its on-chain identity with the integrator-owned object hosting it, and emits no events anywhere in the module. The choice is defensible because any library-side event would need to couple to integrator context

to carry a stable subject ID, but it is undocumented: the module-level doc block covers operator responsibilities, reconfiguration, and upgrade compatibility, but not observability. An integrator scanning `try_consume` or `consume_or_abort` for the usual emit pattern finds none, and may ship without their own emits, leaving off-chain consumers blind to rate-limit hits, cooldown arming, and reconfiguration.

Consider adding a short observability note to the module-level documentation, stating that the module deliberately emits no events because the limiter has no stable identity of its own, and that emissions for rate-limit hits, cooldown arming, and reconfiguration are the integrator's responsibility at their own entry functions.

Update: *Resolved in pull request #348.*

N-06 Cooldown Deadline Overflow Is Mitigated by Operator Discipline Rather Than Validation

The `new_cooldown` constructor accepts an unbounded `cooldown_ms`. When `available` reaches `0`, the arming branch in `try_consume` sets `cooldown_end_ms = now + *cooldown_ms`. Because `cooldown_ms` is unconstrained at construction, this addition can overflow `u64` for absurd operator inputs; Move's runtime then raises a generic `arithmetic_error` and the limiter fails closed. The module-level "Operator responsibilities" section names this as the operator's duty, so the behavior is documented and safe, but the corner relies on documentation rather than enforcement and surfaces only as an unnamed abort.

Consider asserting `*cooldown_ms <= MAX_U64 - now` at the arming site with a named `ECooldownDeadlineOverflow` before computing `now + *cooldown_ms`. The fail-closed semantics are preserved, but the failure surfaces through an explicit error code instead of the generic `arithmetic_error`, making the corner observable and self-documenting to operators triaging an abort.

Update: *Resolved in pull request #349.*

Conclusion

The `rate_limiter` module provides an embeddable rate-limiting primitive for Sui Move, exposing token bucket, fixed window, and cooldown strategies behind a single uniform interface that integrators store as a field inside their own objects. OpenZeppelin audited this module to assess the correctness of its accrual, window rollover, and cooldown arithmetic, and the soundness of the trust boundary it draws between the primitive and the code that embeds it.

Only six notes were raised, with no high, medium, or low severity findings within the audited scope. They concentrate in two areas: the precision and completeness of the documentation, and consistency with established Move conventions and ergonomics. Examples include `try_consume` aborting on a zero amount despite the non-aborting behavior implied by the `try_` prefix, inconsistent field ordering across the constructors, and documentation gaps such as the retroactive credit that arises when a bucket is reconstructed at a faster rate while preserving its original anchor. The underlying state projection and the structural invariants enforced at construction were not found to be affected.

Because the module is an unopinionated primitive that delegates all authorization and policy decisions to the integrating code, its effective security continues to rest on how integrators gate the functions that mutate limiter state and on the reconfiguration semantics they choose. Addressing the surfaced notes would reduce the chance that an integrator embeds the primitive with a weaker guarantee than intended.

The OpenZeppelin development team is thanked for their responsiveness and willingness to provide technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.